

TRACE-RPG: A Trace-Linked Symbolic Commit Gate for Generated Events in an Interactive Game World

Anonymous Author(s)

Abstract—Large language models can generate expressive quests and non-player-character dialogue, but fluent or schema-valid text does not establish that a proposed event is executable, authorized, or consistent with canonical game state. We present TRACE-RPG, a symbolic commit gate that isolates untrusted proposals from state mutation. A type-strict parser, an externally supplied action policy, and six deterministic state-relative predicates decide every admitted transition; rejected candidates may enter bounded repair, and accepted candidates are revalidated before mutation. The counterexample-guided operator ρ reads only the prior candidate and the typed validation errors, never authoritative state. Unkeyed SHA-256 content links, state-semantic replay, episode-continuity checks, and assignment-complete accounting make the traces auditable within that integrity boundary. Three evidence lanes stay separate: a deterministic offline conformance pilot over one authored world, a single-model live screening extension, and a closed-world KG/ontology link-proposal simulation with no runtime authority. The offline lane establishes mechanism conformance only; at $K=1$ the live lane observed 5/5 guided versus 0/5 blind commits only in a deliberately repairable policy-blind regime and null results elsewhere, so it remains pilot-only. No lane establishes population efficacy, model superiority, player experience, affect, retrieval or memory benefit, or commercial-engine performance.

Index Terms—game artificial intelligence, interactive narrative, symbolic validation, constrained generation, runtime enforcement, reproducibility

I. INTRODUCTION

INTERACTIVE generation is more than a language task. A candidate event may be natural yet require an absent item, assert a fact unknown to its speaker, reveal forbidden quest information, introduce an unauthorized effect, or regress a quest stage. Executable text environments make the distinction between a plausible utterance and a valid transition explicit [1], [2]. Grounded dialogue and personalized quest systems further show the value of quest, entity, and graph context [3], [4], but context does not itself authorize canonical mutation.

Structural constraints solve an adjacent problem: grammar-constrained decoding, query languages, and incremental parsers restrict generated structures [5], [6], [7], yet a syntactically admissible event can still be false relative to the current world. Retrieval and memory supply useful evidence [8], [9], [10], but retrieved content is not a state-transition authority either.

This paper presents TRACE-RPG, an implementation that combines these concerns as an auditable transaction protocol. Its current contributions are:

- 1) **C1—Contracts:** *versioned trust-boundary contracts* for state, policy, candidate, record, and game bridge, with a type-strict parser that rejects ambiguous known-field types and unknown candidate keys at proposal and replay boundaries;
- 2) **C2—Controller:** *a validate-repair-commit controller* with six state-relative predicates, bounded repair, unchanged fallback, and defensive pre-commit revalidation;
- 3) **C3—Audit:** *an audit-evidence layer* using unkeyed SHA-256 content links, state-semantic replay, and episode-continuity checks;
- 4) **C4—Harness:** *an offline conformance harness* with frozen assignment keys, classified terminal failures, and assignment-complete accounting, including failures for which no proposal trace can exist; and
- 5) **C5—Repair:** *a counterexample-guided operator* $\rho(a, E)$ that reads only the prior candidate and structured errors, compared with blind unchanged retry and a state-reading oracle upper bound across frozen repairability classes.

Table I, generated from the machine-checked contribution matrix, binds C1–C5 to their prior-work topics, evidence lanes, and inference ceilings; Table IV defines the lanes. Deterministic parser, state-isolation, repair, fault-injection, and manifest fixtures provide the exact-count evidence for these five implementation contributions.

The lanes are non-interchangeable. The empirical center (**E1**) is a deterministic conformance pilot over one authored world state that measures encoded parser, validation, repair, replay, and accounting behavior, not population efficacy. A five-call-per-cell live screening extension (**E2**) with one hosted proposer is capped at pilot-only by its unpinned revision, uncontrolled seed labels, and constructed states. A closed-world KG/ontology simulation (**E3**) evaluates authored typed-link proposals without implementing runtime retrieval. A retained Godot 4.7.1 policy-mirror trace and render replay (ENG1) add engine-local conformance only; their startup-dominated headless frame samples (98.760–116.667 ms) are not a frame benchmark. There are no participants, affect or runtime-retrieval experiments, live Python–Godot authorization round trips, or end-to-end gameplay performance measurements.

TABLE I
CONTRIBUTIONS, PRIOR TOPICS, EVIDENCE LANES, AND INFERENCE
CEILINGS (GENERATED FROM THE STRUCTURED MATRICES)

ID	Contribution	Prior topics (refs)	Lanes	Inference ceiling
C1	Trust-boundary contracts	T1, T2, T3 (13)	E1, ENG1	encoded-field contract conformance
C2	Validate–repair–commit controller	T1, T8 (11)	E1, ENG1	mechanism behavior on frozen cases
C3	Audit-linked evidence layer	T4, T7, T8 (5)	E1, ENG1	integrity/replay on named fixtures
C4	Assignment-complete harness	T4, T6, T7 (18)	E1	exact designed-case accounting
C5	Counterexample-guided operator ρ	T1, T8, T9 (8)	E1, E2	frozen classes + one regime screen

T1 = grounded game worlds; T2 = retrieval/memory agents; T3 = structured generation; T4 = environments/benchmarks; T5 = player experience; T6 = human/LLM evaluation; T7 = statistics/reproducibility; T8 = planning/runtime interposition; T9 = feedback repair. Lanes are defined in Table IV. All rows are authored-fixture verified; none supports a C-RESULT-* efficacy claim.

II. RELATED WORK AND RESEARCH GAP

A. Interactive Narrative and Game Agents

KNUDGE grounds branching NPC dialogue in quest and entity specifications [3]; knowledge-graph quest generation [4], memory–reflection–planning agents [11], and trainable role-playing agents [12] address complementary aspects of grounded, believable behavior. Player-facing studies evaluate preference and perceived dialogue quality [13], [14], [15], and a 2025 preprint reports role-sensitive stability–improvisation trade-offs [16]; both motivate separating hard admissibility from optional naturalness and player-experience constructs.

Experience-driven generation [17], observer-labelled engagement [18], and reported limits of vision-language engagement inference [19] motivate a future adaptation track; affect remains non-authoritative and absent from the present evidence.

B. Environments and Benchmarks

TextWorld and ScienceWorld expose explicit interactive state [1], [2], MineDojo adds open-ended embodied tasks [20], and GVGA separates agent, game, and content tracks [21]. AgentBench, AgentBoard, SOTOPIA, and BALROG evaluate agents across heterogeneous, process-diagnosed, social, and long-horizon game settings [22], [23], [24], [25]; automated play-style agents complement rather than replace human evaluation [26]. None prescribes a transactional authorization boundary.

C. Constraints, Neuro-Symbolic Validation, and Retrieval

Constrained decoders improve output structure [5], [6], [7]; TRACE-RPG instead treats structure as an input condition to independent state-relative authorization. The same generate-and-verify division is established outside games: LLM-Modulo frameworks let a model propose plans that external sound critics accept or reject [27], and SayCan admits a language-proposed skill only after a state-dependent affordance check [28]. TRACE-RPG instantiates the critic as a deterministic, state-relative commit gate that returns typed counterexamples.

Two direct game-specific comparators span complementary boundaries. PANGeA combines memory, validation, a REST service, and Unity integration, but its documented validation asks an LLM to judge and iteratively refine content against designer rules [29]. IVIE, currently a preprint associated with non-archival ICCG 2026 material, incrementally generates interactive-fiction worlds with symbolic validation [30]. A 2026 system in the same lineage restricts LLMs to selecting authored world-state transformations and reports an exploratory eight-participant study of player expression [31]; it maintains consistency by construction but provides no transactional gate, audit-linked records, or repair-operator evidence. Neither comparison implies superiority: each reinforces that validity is only validity for the objectives and predicates its system encodes.

G-Retriever, HippoRAG, and Mem0 address subgraph retrieval and long-term memory [8], [9], [10]; Generative Agents and Character-LLM address behavioral memory and persona [11], [12]. TRACE-RPG may consume such context but denies it commit authority; their benefits are not measured here.

D. Auditability and Reproducibility

Process metrics motivate retaining intermediate failures [23]; robust evaluation cautions against overinterpreting sparse stochastic runs [32]. Reproducibility and environmental reporting motivate frozen artifacts and explicit measurement boundaries [33], [34]. Human and LLM evaluation require construct, rater, and bias controls [35], [36], [37], [38], while future non-inferiority and participant analyses require justified margins and random-effects structures [39], [40].

The bounded novelty is the implemented combination, not the invention of its individual parts. Action interposition is long established. Narrative mediation intercepts a user or agent action before it executes and either blocks it or accommodates it by revising the plan [41]; classical planning defines applicability through preconditions and effects [42]; runtime enforcement wraps an untrusted actor in a monitor that admits only permitted transitions [43]; transaction processing defines the commit/abort discipline under which a failed transaction leaves the prior state intact [44]; authored-logic interactive drama governs which utterances a character may make, in this journal’s predecessor title [45]; and belief-aware narrative planning reasons over what characters know, which is the concern our knowledge and disclosure predicates encode far more coarsely [46]. TRACE-RPG does not claim to have invented interposition. C1–C2 place strict contracts and a validate–repair–commit controller over this lineage; C3–C4 combine checksum-linked terminal records, state-semantic replay, episode continuity, and assignment-complete accounting that retains cases for which no proposal trace can exist; C5 isolates a repair operator that consumes only typed pre-execution validator errors. We do not infer that a feature absent from a report is absent from every implementation.

III. PROBLEM DEFINITION AND GUARANTEE BOUNDARY

For exposition, the controller input used by validation at step t is decomposed as

$$c_t = (G_t, q_t), \quad (1)$$

where G_t denotes typed snapshot fields and q_t denotes the externally authored action-policy, disclosure, and quest-stage constraints stored with that snapshot. Terminal record history $h_{<t}$ is maintained separately from the WorldState snapshot; no validation predicate reads it, and the implementation does not reconstruct state from the log. Retrieval, narrative scores, model rationales, memory summaries, and any future affect estimate are non-authoritative proposal context.

An untrusted proposer emits candidate a_t . Strict parsing first enforces the shared candidate-key contract, including rejection of unknown top-level fields, and establishes the declared schema. The validator returns

$$V(c_t, a_t) = (v_{\text{policy}}, v_{\text{pre}}, v_{\text{reach}}, v_{\text{know}}, v_{\text{disc}}, v_{\text{quest}}, E_t), \quad (2)$$

where E_t is a structured error set. Mutation is

$$c_{t+1} = \begin{cases} T(c_t, a_t), & \bigwedge_i v_i = 1, \\ c_t, & \text{otherwise.} \end{cases} \quad (3)$$

The controller asserts the following implementation invariants under its encoded contracts; the pilot tests their mechanisms with deterministic valid, invalid, repair, fallback, replay, and fault-injection fixtures rather than presenting general theorems. **I1 (exhausted-failure immutability)**: if no returned candidate through budget K validates, or proposal/controller execution fails before commit, the returned state equals the supplied prior state. **I2 (bounded recorded attempts)**: provided every proposal and repair callback returns, repair budget $K \geq 0$ admits at most $K + 1$ recorded candidate attempts; a successful commit adds one defensive validation immediately before mutation. The runtime does not enforce callback deadlines, so I2 is not a wall-clock termination guarantee. **I3 (effect authorization)**: an accepted candidate contains no declared fact effect in `CandidateAction.effects` outside `ActionPolicy.allowed_effects`, and any quest-stage target is explicitly authorized by that policy. **I4 (state-semantic replay)**: with the frozen schemas and deterministic transitions, an accepted trace reconstructs its stated terminal state.

These invariants assume correct policy authorship, faithful extraction into declared candidate fields, deterministic software versions, returning callbacks, and single-process file ownership. Replay revalidates the recorded structured candidate and transition; it does *not* verify the provenance, prompts, or stochastic generation of intermediate repairs, and natural-language implications omitted from structured fields remain outside the guarantee.

IV. TRACE-RPG ARCHITECTURE AND ALGORITHMS

A. Trust Boundary and Contracts

Fig. 1 places the model or recorded adapter outside the canonical owner. The parser rejects ambiguous numeric and

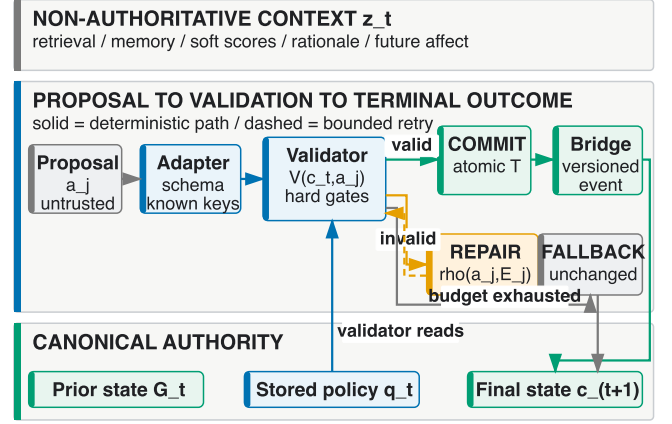


Fig. 1. Trust boundary. Retrieved context and generated candidates remain non-authoritative until known-field type checks and state-relative validation complete.

TABLE II
ENCODED VALIDATOR BOUNDARY

Predicate	Rejects	On failure
Action policy	unknown action/type	unchanged
Precondition	absent/false requirement	unchanged
Reachability	inaccessible object	unchanged
NPC knowledge	undeclared known fact	unchanged
Disclosure	forbidden fact	unchanged
Quest	ineligible/regressive stage	unchanged

collection types for known fields and unknown top-level candidate fields under the shared key contract; the action policy is supplied separately and defines required preconditions and allowed effects; the versioned game bridge carries observations, proposals, validations, rejections, fallbacks, or commits, and only a valid commit authorizes canonical mutation. This decouples interfaces without demonstrating commercial-engine portability. Table II summarizes the encoded rejection boundary; it is not a claim of semantic completeness.

B. Validate–Repair–Commit

Fig. 2 gives the controller’s dependency-free pseudocode. “Record” denotes an append to the in-memory trace representation; persistent writing occurs only after terminal consistency checks.

Validation errors are exposed to a repair callback as structured error codes. Repair never directly mutates state, and exhaustion returns an unchanged deterministic fallback. Fig. 3 shows terminal paths. The pilot compares two implemented callbacks. The *counterexample-guided operator* consumes only the prior candidate a and the validator’s structured error set E , never the authoritative state:

$$\rho(a, E) = a \oplus \bigcup_{e \in E} \delta(e), \quad (4)$$

where each $\delta(e)$ is the deterministic per-error-class edit of Table III and $\oplus$ applies the resulting field edits. Each edit touches at most one candidate field per error entity, so the number of changed fields per attempt is bounded by $|E|$;

```

1:  $a \leftarrow \text{PROPOSE}(c); j \leftarrow 0$ 
2: repeat
3:    $(ok, E) \leftarrow V(c, a); \text{RECORD}(a, E)$ 
4:   if  $ok$  then
5:     assert  $V(c, a).ok$ ; return  $\text{COMMIT}(c, a)$ 
6:   if  $j = K$  then return  $\text{FALLBACK}(c)$ 
7:    $a \leftarrow \text{REPAIR}(c, a, E); j \leftarrow j + 1$ 
8: until terminal

```

Fig. 2. Validate–repair–commit procedure. At most $K + 1$ candidate attempts are recorded; the successful path performs a defensive validation before mutation.

TABLE III
PER-ERROR-CLASS EDITS OF THE GUIDED OPERATOR ρ

Validator error class	Edit $\delta(e)$ on the candidate
Policy precondition omission	insert the named predicate
Policy effect omission	insert the named effect
Policy effect violation	drop the named effect
Unsatisfied precondition	drop the named predicate
Unauthorized stage mutation	clear <code>quest_stage_effect</code>
Unknown action type	no-op (no registered alternative)
Unreachable object; knowledge, disclosure, and stage classes	no-op (candidate-local edit would launder or needs state)

applying ρ twice with the same error set is a fixed point; and an entity simultaneously demanded and rejected on the same field is declared irreparable and left untouched. The *state-reading reference callback* instead discards the error set and reconstructs the policy-governed fields (preconditions, effects, and both quest-stage fields) directly from the authoritative state; it deliberately never edits declaration-integrity fields (required objects, used facts, disclosed facts), because undeclaring a dependency or disclosure would launder the violation past the gate rather than repair the candidate. It is an oracle upper bound over state-readable repairs, and its outcomes must not be read as evidence about any deployable repair method.

C. Integrity, Semantic Replay, and Accounting

Each result row carries an unkeyed SHA-256 *integrity checksum*, and proposal outcomes link to a trace checksum; these detect accidental or tested content changes but are not signatures, writer identity, or protection against an actor who rewrites content and checksums. Semantic replay strictly parses the prior state and every recorded candidate/validation pair, requires every nonfinal attempt to be invalid, recomputes the terminal transition, compares the stated final state, and requires continuity between adjacent JSONL records; it does not re-execute the repair step that produced candidate $j + 1$.

The terminal record classes are commit, symbolic fallback, adapter failure, and controller failure. A controller-failure row remains in the frozen assignment denominator and leaves state unchanged, but it may legitimately lack a completed proposal trace because no candidate outcome was produced. Accordingly, result completeness does not imply trace existence for every terminal row. The pre-execution assignment key contains run, arm, scenario, seed, model, revision, controller-configuration hash, assignment-input hash, and prior-state hash. Aggregation rejects duplicate, missing,

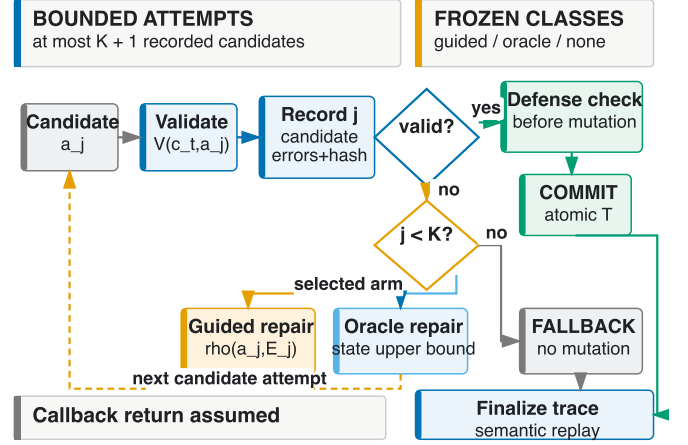


Fig. 3. Proposal, validation, bounded repair, fallback, commit, and replay states. Controller failure is terminal and may have no completed proposal trace.

TABLE IV
EVIDENCE LANES AT A GLANCE: DESIGN, UNIT, COMPARISON, AND CEILING (GENERATED FROM THE EXPERIMENT MATRIX)

Lane	Design	Unit / budget	Comparison	Ceiling
E1	Offline conformance: authored frozen fixtures, one world	13 gate; 12 repair/arm; 10 faults; 7 adapter; 3 guards	reject vs blind vs ρ vs oracle	mechanism conformance
E2	Live RQ2 screening: one hosted proposer, matched candidate	5 cells $\times$ 5 calls; $K=1$	ρ vs blind per regime	pilot-only transfer screen
E3	KG/ontology simulation: closed-world typed-link holdout	43 nodes; 106 ref. edges; 24 typed; 210 scores	degree baseline vs 6 fixed strategies	construction result only
ENG1	Godot/Web engineering: engine-local policy mirror, no participants	4 fixtures; 49 checks; 8 smoke; 5 archetypes	authored fixture paths	engine-local conformance

Lanes are non-interchangeable with different denominators; none establishes population efficacy, model ranking, player experience, retrieval benefit, or commercial-engine performance.

or unexpected keys and separately reports symbolic, adapter, and controller failures.

V. EVIDENCE LANES AND BOUNDARIES

Table IV fixes each lane’s design, unit, comparison, and ceiling before any count is read.

A. E1 Offline Conformance Design

The pilot asks whether (PQ1) valid candidates commit while named invalid-policy cases remain unchanged; (PQ2) rejection-only, blind unchanged retry, ρ , and the state-reading reference callback produce the expected outcome in every frozen repairability class; (PQ3) frozen checksum, state-semantic, linkage, and continuity mutations are rejected by their designated check operations and an injected second-file append failure triggers the pair-write rollback check; (PQ4) strict adapters and assignment manifests retain exactly one classified terminal row per assigned case or fail closed; and (PQ5) proposal and replay parsers both reject an otherwise-valid candidate carrying one unknown top-level key.

Cases are purposively designed conformance fixtures in four domains (validator/state isolation, repair-arm comparison, record/trace fault injection, adapter/accounting contracts), not samples from a model or game population. Every repair fixture declares a frozen repairability class before execution: *guided-repairable* (the error payload alone suffices for ρ), *oracle-only* (repair needs the authoritative state), or *irreparable* (no arm may commit); each class’s expected outcome per arm is part of the fixture contract, unique cases form the denominators, and all expectations precede execution.

B. E1 Measures and Frozen Artifacts

Measures are exact counts: validator-class agreement; rejected or failed cases with state mutation; commit and attempt outcomes by arm and repairability class; detectable faults rejected by their designated operation; replayed traces; rejected manifest violations; and proposal/replay parity on the unknown-key regression, which is parser-contract parity rather than a safety outcome. Exception class alone does not identify a stable detector layer, so no detector attribution is reported, and no p -values or population intervals are attached. Adapter cases carry JSON-Schema-pinned synthetic latency and token constants that exercise accounting propagation only. The release packet freezes fixture and assignment manifests, schemas, the result JSON with embedded traces, environment and code revision, generated tables, and checksums, following reproducibility and measurement-boundary guidance [33], [34].

C. E2 Live RQ2 Screening

The live extension compares blind unchanged retry with ρ at matched $K=1$: per cell, five seed-indexed Codex CLI calls request one candidate from `gpt-5.6-sol`, and the same candidate feeds both arms, eliminating within-call proposal differences. Conditions vary policy visibility and whether a trivial zero-effect route exists. The promoted packet retains the current cells and the superseded v1 diagnostic, binds every JSON/JSONL receipt by SHA-256, and labels the evidence `screening-pilot-only`. Because seed labels do not control the hosted sampler, the revision is unpinned, token accounting is unavailable, and constructed states are not population samples, we report exact counts and failure classes without p -values, confidence intervals, or model ranking.

D. E3 KG/Ontology Link-Proposal Simulation

`SL-KG-ONTOLOGY-SIM-001` is an engineering-only closed-world lane separate from the authoritative `WorldState`: 43 OKF nodes and 106 reference edges, 24 reviewed typed edges, passing type/domain/range checks and six competency questions, with six authored queries scored under seven fixed strategies. The retained `S2-typed-lexical-loose` strategy recovered 6/6 authored holdouts ($P = R = F_1 = 1.000$, realistic-tie $MRR = 0.944$, Brier = 0.131, Sem@3 = 1.000); the degree baseline recovered none. These are construction results, not OWL/SHACL conformance, semantic completeness, runtime retrieval, or support for any `C-RESULT-*` claim.

VI. PILOT RESULTS

A. Declared-Field Gate Conformance

The pilot loader admits a fixture set only when it isolates every implemented validator code exactly once alongside one valid control, so the 13/13 agreement and the observation of all 12 implemented error codes are *construction invariants of the harness*, not measured success rates. What the run adds is the part the loader does not enforce: each of the 12 isolated negative fixtures showed observed agreement with its authored expected code rather than reaching a different one, and every noncommit path preserved the complete canonical state. This is implementation conformance to an authored structured-field oracle over a single world state, not accuracy against independent semantic labels.

B. Repair-Arm Comparison on Designed Fixtures

The 12 initially invalid designed cases are partitioned into frozen repairability classes: 5 guided-repairable, 1 oracle-only, and 6 irreparable. Rejection-only and blind unchanged-candidate retry each committed 0/12. The counterexample-guided operator ρ committed 5/5 guided-repairable cases and, by design, 0/1 oracle-only and 0/6 irreparable cases; every ρ commit edited a mean of 1.0 candidate fields, and every failed arm execution left the canonical state unchanged. The state-reading reference callback committed 5/5 guided-repairable and 1/1 oracle-only cases, with 0/6 irreparable. These exact counts separate the three mechanisms on the designed fixtures: blind retry never converts a counterexample into a commit, ρ converts exactly the cases whose error payload carries sufficient information, and the oracle additionally converts the case requiring state knowledge. The 5 committed guided cases initially carried `POLICY_PRECONDITION_OMISSION+UNSATISFIED_PRECONDITION`, `POLICY_EFFECT_OMISSION`, `POLICY_EFFECT_VIOLATION`, `UNSATISFIED_PRECONDITION`, `POLICY_QUESTION_STAGE_EFFECT_VIOLATION`, respectively, and each became valid through the corresponding Table III edit $\delta(e)$ alone. This is operator conformance on frozen fixtures, not repair quality of any live model.

C. Integrity, Boundaries, and Assignment Accounting

All 10 faults prespecified as detectable were rejected by their designated check operations (10/10). Because this pilot does not compare stable typed detector codes, it does not establish detector-layer attribution. Separately, the known provenance-boundary fixture substituted a different invalid precursor before the same recorded valid repair, recomputed the checksum, and passed replay as expected (1/1 undetected). Thus, the mechanism does not protect against a writer who can recompute unkeyed checksums, and replay does not authenticate the repair-generation operation. Two open boundary sentinels—unextracted narrative disclosure and simultaneous omission of a required object from candidate and policy—were intentionally accepted at the encoded layer (2/2), with 0 labelled as safety passes. The former unknown-top-level-field acceptance boundary was reclassified after Stage 8 as a



Fig. 4. Status/event crops from the authored Godot render replay with manifest metadata; trace-bound correspondence only, not live-transport, performance, usability, or efficacy evidence.

Panel	Event	Validation	State
(a)	evt-000-observe	n/a	= before
(b)	evt-002-fallback-secret	invalid	= before
(c)	evt-005-commit-hint	valid	≠ before

Captures sl-rc-001, -002, -003 (.png) under godot-4.7.1-20260813t115916z-sealed-lighthouse-render-v5/rendered-canonical-v1/.

TABLE V
EXACT REPAIR-ARM COUNTS BY REPAIRABILITY CLASS (DESIGNED FIXTURES)

Arm	Info source	Commits / cases			Mean edits ^e
		G-rep. ^f	O-only ^f	Irrep. ^f	
Rejection-only ($K=0$)	none	0/5	0/1	0/6	—
Blind unchanged retry	none	0/5	0/1	0/6	—
Counterexample-guided ρ	error set E	5/5	0/1	0/6	1.0
State-reading oracle	state+policy	5/5	1/1	0/6	1.0

^eMean changed candidate fields over committed repairs (minimality proxy); failed arm executions left the state unchanged. ^fCommits/cases. G-rep. = guided-repairable, O-only = oracle-only, Irrep. = irreparable (frozen repairability classes).

TABLE VI
FROZEN-PILOT CHECKS AND TERMINAL OUTCOMES (ROWS ARE NOT COMPARABLE)

Check or terminal family	Exact outcome	Interpretation
Gate fixture agreement	13/13	authored-oracle agreement
Known-boundary acceptance	2/2	encoded limit, not a safety pass
Closed unknown-key rejection	1/1	proposal/replay parser parity
Detectable integrity-fault rejection	10/10	designated check operations
Known provenance-boundary acceptance	1/1	repair provenance unauthenticated
Adapter terminal classes	1 commit; 1 fallback; 5 failures / 7 assigned	complete classification of 7 assignments
Manifest-guard rejection	3/3	injected faults failed closed

Rows have different denominators and favorable directions; they must not be ranked as rates. Every value is an exact audit fact for designed fixtures.

closed negative regression: both proposal and replay parsers specifically rejected a complete 12-field candidate carrying one unknown key (1/1). This establishes parser rejection parity, not semantic safety. Among seven adapter/accounting assignments, outcomes were 1 commit, 1 symbolic fallback, and 5 adapter failures. Synthetic telemetry fields were populated and propagated for 2/7 assignments; these are schema-pinned constants verifying accounting-field propagation, not measurements. All 3/3 injected duplicate or missing-assignment guards failed closed. These are raw counts from synthetic offline fixtures.

D. Live Screening Pilot (Pilot-Only)

A separate screening pilot called gpt-5.6-sol through the Codex CLI. It used 5 seed-indexed proposal calls per cell and repair budget $K = 1$; within each call, blind unchanged retry

TABLE VII
LIVE SCREENING PILOT, EXACT COUNTS (FIVE PROPOSALS PER CELL, $K=1$)

Base / condition	Initial valid	ρ vs blind commits	Δ	Initial errors
Frozen / visible	5/5	5/5 vs 5/5	+0	—
Frozen / blind	5/5	5/5 vs 5/5	+0	—
Frozen / goal-blind	0/5	0/5 vs 0/5	+0	QSR
Signal-v2 / visible	5/5	5/5 vs 5/5	+0	—
Signal-v2 / blind	0/5	5/5 vs 0/5	+5	PEO+PEV+PPO

QSR = quest-stage regression, PEO = policy-effect omission, PEV = policy-effect violation, PPO = policy-precondition omission. Δ is ρ commits minus blind commits for matched candidates. All counts are screening-pilot-only, without inferential statistics; the superseded v1 diagnostic is excluded.

and ρ received the same initial candidate. The hosted revision was unpinned, token accounting was unavailable, and the seeds did not control the hosted sampler, so the calls are quasi-replicates rather than independent randomized samples. We report neither inferential statistics nor a model ranking.

In Table VII, a guided advantage appeared only in the policy-blind signal-repair-v2 cell, a state deliberately constructed to elicit guided-repairable errors. All 5/5 initial candidates were invalid; ρ committed 5/5, whereas blind retry committed 0/5. The other four current cells showed no guided advantage: three had already-valid initial candidates, and the frozen-base goal-directed cell produced irreparable quest-stage regressions. All 15/15 noncommit arm outcomes in the current cells preserved the prior-state hash. This supports mechanism transfer only in that error regime, not population efficacy, model promotion, or a general sample-efficiency claim. Accordingly, C-PILOT-007 and C-PILOT-008 are pilot-only while C-RESULT-003 remains TODO-RESULT. In the superseded v1 diagnostic, guided repair reduced up to 3 co-occurring errors to one irreparable knowledge error but still could not commit, showing that partial per-error repairs do not compose when any residual error is irreparable.

Fig. 5 makes the offline reporting boundary explicit: only rows admitted by the frozen fixture and designated check operation enter the offline tables, whereas the live table arrives through the hash-bound screening promotion manifest and never enters the offline denominator.

ENG1. Godot 4.7.1 replayed the retained *Sealed Lighthouse* canonical trace in a non-headless 1280×720 SubViewport; its manifest binds each PNG to event/delivery indices, validation, and state hashes, so Fig. 4(b) preserves the before hash whereas (c) changes it after a valid commit. The crops enlarge only authored status/event callouts of generated-asset-free renders. The v5 packet establishes one scripted fixture's render/state correspondence only; it adds no live model or

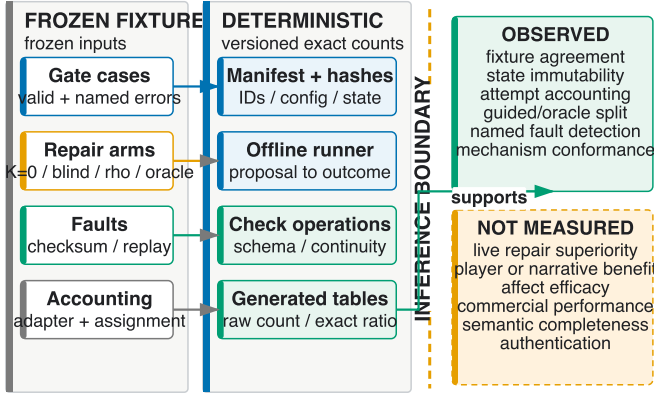


Fig. 5. Evidence boundary from frozen fixture through a designated check operation and generated table. Counts are admitted only through generated artifacts.

Python transport, participant input, adaptation intervention, or narrative-quality result.

VII. DISCUSSION

A. Established Scope

The designed pilot establishes mechanism behavior only for its frozen cases: encoded checks isolate state, bounded repair and fallback follow their declared paths, named corruptions are rejected by their designated operations, and assigned rows remain classified. It does not attribute a rejection to a stable detector code. This suffices for authoring-time validation, CI regression, recorded-adapter tests, and pre-engine replay inspection.

The guided-versus-oracle boundary is the honest reading of the offline repair comparison: the state-reading callback may consult authoritative state and so converts the oracle-only case that ρ by construction cannot, while ρ matches the oracle exactly on the guided-repairable class using only the validator’s counterexamples. The live extension separated ρ from blind retry only when the constructed state placed every proposal in the guided-repairable class; already-valid and guided-irreparable regimes were null. This regime dependence, the unpinned hosted revision, and quasi-replicated calls block any general sample-efficiency or model-capability claim. Feedback-consuming repair loops are precededented: Self-Refine feeds back language generated by the same untrusted model [47], Reflexion stores verbal self-reflections across trials [48], and Self-Debugging consumes execution results from a trusted full-program runtime [49]. ρ differs in channel and authority: its feedback is a typed error set computed by the deterministic validator before execution, and every failed live arm remains state-isolated. Its name follows counterexample-guided inductive synthesis, in which a verifier’s counterexample rather than the full specification state drives the next candidate [50]; here the verifier is the commit gate itself.

B. Relationship to Prior Work

TRACE-RPG applies state-relative authorization after parsing, complementing constrained decoders [5], [6], [7]; with-

holds commit authority from retrieval and memory components [8], [9], [10]; and complements rather than replaces executable environments and benchmarks [1], [2], [22], [25]. Process-linked traces instantiate a measurement concern emphasized by AgentBoard [23], not evidence that TRACE-RPG outperforms any system.

The studies that would test confirmatory efficacy—a pre-registered multi-model comparison with an independently labelled semantic oracle and a genuine blind-retry comparator, retrieval and memory ablations at matched encoded validity, and a blinded matched-validity human study with justified power and mixed-effects analysis—remain outside this paper. The screening pilot does not substitute for any of them [22], [35], [32], [40].

VIII. THREATS, ETHICS, AND ARTIFACT SCOPE

The evidence has nine explicit boundaries. **L1**: encoded predicates cannot detect semantics omitted from structured fields. **L2**: one authored world and one constructed live state do not establish cross-world, cross-policy, or scale generalization. **L3**: the state-reading callback is an oracle; offline classes and regime-dependent screening do not establish general repair quality or sample efficiency. **L4**: the single hosted proposer has an unpinned revision, no token accounting, and uncontrolled seeds; no live Python–Godot round trip exists. **L5**: no participant or affect data exist. **L6**: unkeyed hashes give integrity, not writer authentication or tamper resistance. **L7**: record consistency assumes one writer. **L8**: engine evidence is one Godot fixture with startup-dominated samples, not a portability or real-time result. **L9**: selected fixtures and screening cells support no population, safety-rate, or model-ranking inference. Internal validity also depends on independent policy expectations and no fixture leakage.

The pilot processes no participants or personal telemetry; future human and affect work requires ethics review, informed consent, minimization, retention rules, and opt-out. LLM judges, if used, remain auxiliary because automated and crowd judgements both require calibration and bias controls [35], [36], [37], [38].

IX. CONCLUSION

TRACE-RPG isolates candidate-event payloads from canonical mutation through an encoded symbolic commit gate and makes terminal outcomes auditable through linked records and state-semantic replay. The offline pilot establishes mechanism conformance on frozen fixtures; the live screening pilot separated ρ from blind retry (5/5 versus 0/5) only in the constructed repairable regime and remains pilot-only. Cross-model superiority, player-experience benefit, retrieval or memory effectiveness, affect efficacy, and commercial-engine performance remain confirmatory extensions.

DISCLOSURE OF AI USAGE

Large language models assisted prose, code and test drafting, command orchestration, evidence review, simulated peer review, and citation-identity checks; a hosted model produced

the separately labelled live-screening candidates. Every reported count comes from deterministic runners or hash-bound receipts. The authors cross-checked claims against source artifacts and take full responsibility for the content.

DATA AND CODE AVAILABILITY

The repository contains the implementation, build sources, the separated offline and live-screening packets, and the machine-checked contribution-evidence-matrix.csv, reference-topic-crosswalk.csv, and experiment-evidence-matrix.csv from which Tables I and IV are generated. The offline manifest binds 38 artifacts, 22 inputs, and 121 provenance rows; the live promotion manifest binds 14 JSON/JSONL receipts. Clean-Git input binding is re-established at tag trace-rpg-guided-repair-inputs-20260821-v1. No reviewer archive or DOI deposit is claimed.

REFERENCES

- [1] M.-A. Côté, Á. Kádár, X. Yuan *et al.*, “TextWorld: A learning environment for text-based games,” in *Computer Games*, ser. Communications in Computer and Information Science. Springer International Publishing, 2019, vol. 1017, pp. 41–75.
- [2] R. Wang, P. Jansen, M.-A. Côté, and P. Ammanabrolu, “ScienceWorld: Is your agent smarter than a 5th grader?” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2022, pp. 11 279–11 298.
- [3] N. Weir, R. Thomas, R. d’Amore, K. Hill, B. Van Durme, and H. Jhamtani, “Ontologically faithful generation of non-player character dialogues,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 9212–9242.
- [4] T. Ashby, B. K. Webb, G. Knapp, J. Searle, and N. Fulda, “Personalized quest and dialogue generation in role-playing games: A knowledge graph- and language model-based approach,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, 2023, pp. 1–20.
- [5] S. Geng, M. Josifoski, M. Peyrard, and R. West, “Grammar-constrained decoding for structured NLP tasks without finetuning,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 10 932–10 952.
- [6] L. Beurer-Kellner, M. Fischer, and M. Vechev, “Prompting is programming: A query language for large language models,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1946–1969, 2023.
- [7] T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing incrementally for constrained auto-regressive decoding from language models,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 9895–9901.
- [8] X. He, Y. Tian, Y. Sun *et al.*, “G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering,” in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 132 876–132 907.
- [9] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 59 532–59 569.
- [10] P. Chhikara, D. Khan, S. Aryan, T. Singh, and D. Yadav, “Mem0: Building production-ready AI agents with scalable long-term memory,” in *ECAI 2025*, ser. Frontiers in Artificial Intelligence and Applications. IOS Press, 2025, peer-reviewed ECAI 2025 record; page range was not exposed by the accessible Crossref or publisher metadata at audit time.
- [11] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulators of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 2023, pp. 1–22.
- [12] Y. Shao, L. Li, J. Dai, and X. Qiu, “Character-LLM: A trainable agent for role-playing,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 13 153–13 187.
- [13] N. Akoury, Q. Yang, and M. Iyyer, “A framework for exploring player perceptions of LLM-generated dialogue in commercial video games,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 2295–2311.
- [14] M. Hochreiter, S. Kriglstein, and G. Wallner, “Beyond pre-defined scripts: Player perceptions on generative non-player character dialogues,” in *Proceedings of the 31st International Conference on Intelligent User Interfaces*. ACM, 2026, pp. 2004–2018.
- [15] M. Yin, H. Zu, W. Gu *et al.*, “How contextualized generative AI shapes player experience in games,” *Entertainment Computing*, vol. 58, p. 101 194, 2026, online record with a future issue assignment at the 2026-08-12 audit; issue metadata must be rechecked at submission.
- [16] V. Figueiredo and D. Elumze, “Symbolically scaffolded play: Designing role-sensitive prompts for generative NPC dialogue,” 2025, preprint; no archival venue or publisher DOI verified as of 2026-08-12.
- [17] G. N. Yannakakis and J. Togelius, “Experience-driven procedural content generation,” *IEEE Transactions on Affective Computing*, vol. 2, no. 3, pp. 147–161, 2011.
- [18] D. Melhart, D. Gravina, and G. N. Yannakakis, “Moment-to-moment engagement prediction through the eyes of the observer: PUBG streaming on Twitch,” in *International Conference on the Foundations of Digital Games*. ACM, 2020, pp. 1–10.
- [19] Z. Wang, Q. Guo, R. Singh, and X. Hu, “Do vision language models understand human engagement in games?” 2026, preprint; no archival venue or publisher DOI verified as of 2026-08-12.
- [20] L. Fan, G. Wang, Y. Jiang *et al.*, “MineDojo: Building open-ended embodied agents with internet-scale knowledge,” in *Advances in Neural Information Processing Systems*, vol. 35. Neural Information Processing Systems Foundation, 2022, pp. 18 343–18 362.
- [21] D. Pérez-Liebana, J. Liu, A. Khalifa, R. D. Gaina, J. Togelius, and S. M. Lucas, “General video game AI: A multitask framework for evaluating agents, games, and content generation algorithms,” *IEEE Transactions on Games*, vol. 11, no. 3, pp. 195–214, 2019.
- [22] X. Liu, H. Yu, H. Zhang *et al.*, “AgentBench: Evaluating LLMs as agents,” in *International Conference on Learning Representations*, 2024, pp. 52 989–53 046, peer-reviewed official proceedings record; no publisher DOI assigned.
- [23] C. Ma, J. Zhang, Z. Zhu *et al.*, “AgentBoard: An analytical evaluation board of multi-turn LLM agents,” in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 74 325–74 362.
- [24] X. Zhou, H. Zhu, L. Mathur *et al.*, “SOTOPIA: Interactive evaluation for social intelligence in language agents,” in *International Conference on Learning Representations*, 2024, pp. 40 975–41 019, peer-reviewed official proceedings record; no publisher DOI assigned.
- [25] D. Paglieri, B. Cupial, S. Coward *et al.*, “BALROG: Benchmarking agentic LLM and VLM reasoning on games,” in *International Conference on Learning Representations*, 2025, pp. 96 666–96 702, peer-reviewed official proceedings record; no publisher DOI assigned.
- [26] P. Le Pelletier de Woillemont, R. Labory, and V. Corruble, “Automated play-testing through RL-based human-like play-styles generation,” *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 18, no. 1, pp. 146–154, 2022.
- [27] S. Kambhampati, K. Valmeekam, L. Guan *et al.*, “Position: LLMs can’t plan, but can help planning in LLM-Modulo frameworks,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 22 895–22 907.
- [28] B. Ichter, A. Brohan, Y. Chebotar *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Proceedings of the 6th Conference on Robot Learning (CoRL 2022)*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318.
- [29] S. Buongiorno, L. Klinkert, Z. Zhuang, T. Chawla, and C. Clark, “PANGeA: Procedural artificial narrative using generative AI for turn-based, role-playing video games,” *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 20, no. 1, pp. 156–166, 2024.
- [30] M. Vaucher, S. Silveira, S. Góngora, and L. Chiruzzo, “TVIE: A neuro-symbolic approach to incremental and validated generation of interactive fiction worlds,” 2026, preprint; accepted/forthcoming at ICCV 2026, whose available pre-proceedings were explicitly non-archival and unpublished at retrieval time.
- [31] S. Góngora, L. Chiruzzo, G. Méndez, and P. Gervás, “World-state transformations for neuro-symbolic interactive storytelling,” 2026, preprint; no archival venue or publisher DOI verified as of 2026-08-21.
- [32] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, “Deep reinforcement learning at the edge of the statistical precipice,” in *Advances in Neural Information Processing Systems*, vol. 34. Curran Associates, Inc., 2021, pp. 29 304–29 320, peer-reviewed official proceedings record; no publisher DOI assigned.
- [33] J. Pineau, P. Vincent-Lamarre, K. Sinha *et al.*, “Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program,” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021, peer-reviewed official journal article; no publisher DOI assigned.
- [34] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, “Towards the systematic reporting of the energy and carbon footprints of machine learning,” *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020, peer-reviewed official journal article; no publisher DOI assigned.
- [35] C. van der Lee, A. Gatt, E. van Miltenburg, S. Wubben, and E. Krahmer, “Best practices for the human evaluation of automatically generated text,” in *Proceedings of the 12th International Conference on Natural Language Generation*. Association for Computational Linguistics, 2019, pp. 355–368.
- [36] M. Karpinska, N. Akoury, and M. Iyyer, “The perils of using Mechanical Turk to evaluate open-ended text generation,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 1265–1285.
- [37] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, vol. 36. Neural Information Processing Systems Foundation, 2023, pp. 46 595–46 623.
- [38] G. H. Chen, S. Chen, Z. Liu, F. Jiang, and B. Wang, “Humans or LLMs as the judge? a study on judgement bias,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 8301–8327.
- [39] D. Lakens, A. M. Scheel, and P. M. Isager, “Equivalence testing for psychological research: A tutorial,” *Advances in Methods and Practices in Psychological Science*, vol. 1, no. 2, pp. 259–269, 2018.
- [40] D. J. Barr, R. Levy, C. Scheepers, and H. J. Tily, “Random effects structure for confirmatory hypothesis testing: Keep it maximal,” *Journal of Memory and Language*, vol. 68, no. 3, pp. 255–278, 2013.
- [41] M. O. Riedl, C. J. Saretto, and R. M. Young, “Managing interaction between users and agents in a multi-agent storytelling environment,” in *Proceedings of the Second International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2003, pp. 741–748.
- [42] R. E. Fikes and N. J. Nilsson, “STRIPS: A new approach to the application of theorem proving to problem solving,” *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971.
- [43] F. B. Schneider, “Enforceable security policies,” *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.
- [44] T. Haerder and A. Reuter, “Principles of transaction-oriented database recovery,” *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, 1983.
- [45] R. Evans and E. Short, “Versu—a simulationist storytelling system,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 6, no. 2, pp. 113–130, 2014, predecessor title of IEEE Transactions on Games.
- [46] S. G. Ware and C. Siler, “Sabre: A narrative planner supporting intention and deep theory of mind,” in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, vol. 17, no. 1, 2021, pp. 99–106.
- [47] A. Madaan, N. Tandon, P. Gupta *et al.*, “Self-Refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 46 534–46 594.
- [48] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 8634–8652.
- [49] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching large language models to self-debug,” 2023, preprint; no archival venue or publisher DOI verified as of 2026-08-21.
- [50] A. Solar-Lezama, L. Tancu, R. Bodík, S. Seshia, and V. Saraswat, “Combinatorial sketching for finite programs,” in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2006, pp. 404–415.